

Configuration Recognition Via Border Tracing

Jad Bassil

1 Objectives

1. Determine a memory-efficient representation of 2D lattice-based modular robots through a distributed border tracing message-passing algorithm. The algorithm must account for holes inside the structure.
2. Understand and implement an algorithm given its description and pseudo-code.
3. Provide a complexity analysis of your algorithm.
4. Validate and evaluate the algorithm on different configuration.

2 Work to return for evaluation

Read and implement the border tracing algorithm described in 3 in *VisibleSim* with the *Blinky Blocks* modules. Test it on the several configurations and upload the code, a description of your work and results to moodle. You must include:

1. The list of members of the group (2 people max),
2. Your source directory,
3. Your configuration xml files,
4. Screenshots showing the different configurations and found borders.
5. Figure 2 shows an example where the number of elements in the representation can be minimized by considering collinear modules. Propose, implement and describe your own solution to further optimize the representation.
6. Report the number of messages and simulator time for each tested configuration.
7. Provide a time, memory and communication complexity.

Your report must be written in LaTeX.

3 Algorithm Description

Our robots are placed in a square 2D grid, so each cell has 4 neighbors connected by its border. Since we're considering 4 connections, modules are part of the border if they have at least one empty direct neighbor cell. Two types of border can appear: external borders and internal borders, which indicate a hole in the configuration.

In this section, we describe the distributed algorithm for determining the set of borders in a given configuration. After the borders are identified, their representation can be compressed and stored in each module's memory. This data allows each module to locally verify whether a given position is part of the current configuration. The verification is performed using the ray-casting algorithm, which counts how many times a ray, originating from the given position and extending

in a predetermined direction, intersects with the borders. A position outside the configuration will result in an even number of ray-border intersections. A position inside the configuration will result in an odd number of intersections.

In the next sections, a distributed algorithm is described to search for borders. The system assumptions are given in section 3.1. The algorithm description consisting of messages tracing each border to determine an efficient border representation is provided in section 3.2.

3.1 System assumptions

In this work, it is assumed that the modular robot system has the following properties:

- Modules are positioned on the cells of a regular 2D square lattice, where each module has local knowledge of its coordinates and orientation.
- Communication is restricted to neighbor-to-neighbor message-passing. A module can send messages to its adjacent neighbors via one of its connectors, and the receiving module can identify the direction of the sender.
- Each module's perspective is limited to its immediate neighborhood, with computations performed locally based solely on information obtained through message-passing within the neighborhood.
- Each module is aware of its direct connections, specifically identifying which of its borders are connected to neighboring modules and which are not.
- The configuration is assumed to remain fixed and fully connected throughout the process. No modules are added or removed during the algorithm's execution.

3.2 Borders Definition

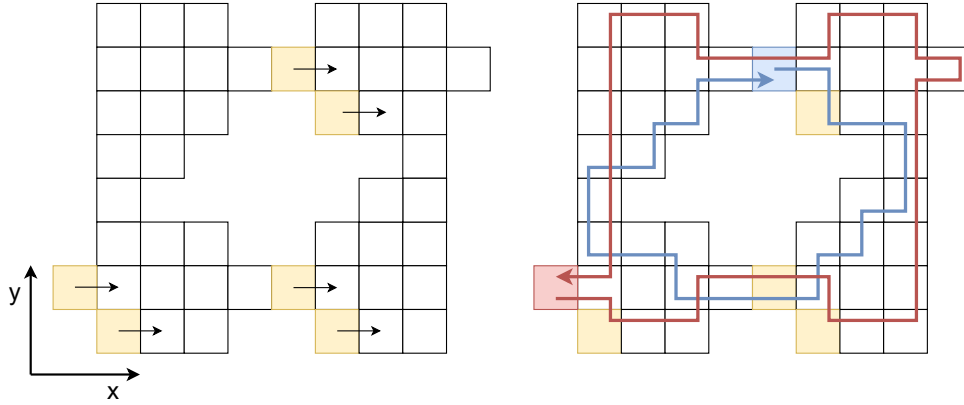


Figure 1: An example of initiators and two paths following borders. Initiators are shown in yellow. The red and blue arrows shows the only two completed paths starting and ending at the red and blue initiators.

Algorithm 1 describes a distributed algorithm to search for borders. The algorithm is initiated by modules satisfying the following condition (cf. Algorithm 1 lines 1-2):

$$initiator(x, y) = empty(x, y - 1) \wedge [empty(x - 1, y) \vee \neg empty(x + 1, y - 1)] \quad (1)$$

This condition ensures that all borders internal and external will have at least one initiator. All initiators will initiate a message (*BORDER_SEARCH_MSG*) that will follow the border (cf. Algorithm 1 lines 11-13). Fig. 1 (left) shows all initiators modules in yellow. The following of the

Algorithm 1: Borders search

```
input :  $P$  // Module's position
         $directions \leftarrow \{FRONT, RIGHT, BACK, LEFT\}$  // Possible movement
        directions
         $dirInit$  // Key-value pairs where keys corresponds to received
        direction and values to initiators positions
output:  $border$  // The border's information. A set of corners' coordinates.

1 Function  $isInitiator()$ :
2   | return  $empty(x, y - 1) \wedge (empty(x - 1, y) \vee \neg empty(x + 1, y - 1))$ 
3 Function  $getNextDir(prevDir \in directions)$ :
4   |  $k \leftarrow (prevDir + 3) \bmod 4$ 
5   | for  $i \in [0, 3]$  do
6   |   |  $Q \leftarrow P + nextPositionInDir(P, k)$  //  $Q$  gets the position of the neighbor in
6   |   | direction  $k$ 
7   |   | if  $(\neg empty(Q))$  then
8   |   |   | return  $k$ 
9   |   |  $k \leftarrow (k + 1) \bmod 4$ 
10 Initialization:
11 | if  $isInitiator()$  then
12 |   |  $nextDir \leftarrow getNextDir(FRONT)$ 
13 |   | send  $BORDER\_SEARCH\_MSG(nextDir, border \cup P, P)$  to  $nextDir$ 
14 Msg Handler  $BORDER\_SEARCH\_MSG(prevDir, border, initiatorPos)$ :
15 | if  $(P = initiatorPos)$  then
15 |   | // border found
16 |   | return
17 | if  $(prevDir \in dirInit \wedge dirInit[prevDir] < initiatorPos)$  then
18 |   | return // already received a message from  $prevDir$ 
19 |  $nextDir \leftarrow getNextDir(prevDir)$ 
20 | if  $(nextDir \neq prevDir \wedge isOnBorder())$  then
20 |   | // Is a corner
21 |   |  $border \leftarrow border \cup P$ 
22 |  $dirInit[prevDir] \leftarrow initiatorPos$ 
23 | send  $BORDER\_SEARCH\_MSG(nextDir, border, initiatorPos)$  to  $nextDir$ 
```

border is done according to the $getNextDir$ functions that returns the next direction to follow (cf. Algorithm 1 lines 3-9). Each module records the initiator's position for every received direction to track ongoing border searches and to allow participation in multiple borders.

The message will proceed along its path if the receiving module has not yet received the message from the sender's direction, or if the initiator's position is smaller than the previously recorded initiator position. A position (x_1, y_1) is defined as less than another position (x_2, y_2) if and only if the eq.2 is verified.

$$(x_1, y_1) < (x_2, y_2) \iff [(x_1 < x_2) \vee ((x_1 = x_2) \wedge (y_1 < y_2))] \quad (2)$$

This ordering prioritizes the x -coordinates and uses the y -coordinates as a tiebreaker when x -coordinates are equal. This guarantees that only the initiator with the minimum position receives the message after a complete traversal of the border. Consequently, this marks the termination of the border-following process, ensuring that all border information is collected by that module (cf. Algorithm 1 lines 15-18). The initiator can then broadcast the border representation to all the system.

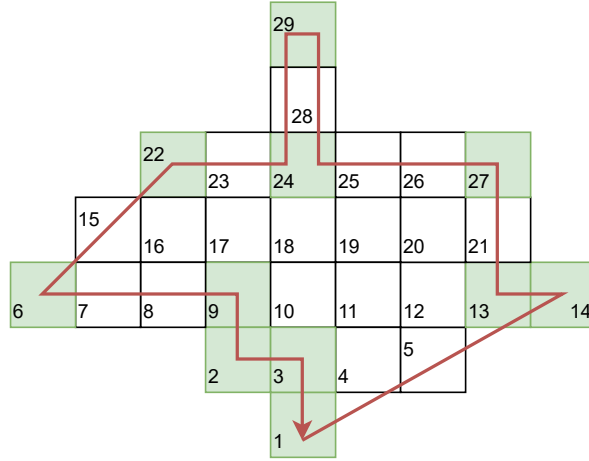


Figure 2: An example of a border found using Algorithm 1. The green cells represent modules whose coordinates are border's corners. The border segments are shown in red. The two Segments formed by modules 1,5 and 14 and 22, 15 and 6 satisfies the collinearity conditions. Note that a segment is not formed between 9,2,3 and 1 since one more corner is required.

The border information is expressed by the sequence of corners coordinates, each encoded in two bytes, as shown in Algorithm 1 lines 20-21.

These corners are found through the message-passing border traversal. At each change in message direction, the position P_m of module m is added to the border if m belongs to a border. This can be checked locally by verifying that the number of neighbors is less than four cf. Algorithm 1 line 20-21): A border example found by the described algorithm can be seen in Figure 2.